COMPILER DESIGN LABORATORY · BCSE307P

SwayaLang

A Domain-Specific Language & Compiler for Stock-Trading Strategies

GitHub Repository: https://github.com/shuklaswayam/SwayaLang

• PHASE 2: CORE IMPLEMENTATION

• 63/63 TESTS PASSING

STUDENT	REG. NO.	INSTITUTION
Swayam Shukla	24BCE0751	SCOPE, VIT Vellore

COMPLETE COMPILER PIPELINE OPERATIONAL

Source (.strat) → Tokens → AST → Scoped SymTab → Semantics → TAC Quadruples → Interpreter & Backtest

examples/multi_rule.strat

```
# MultiRule — risk management with STOP_LOSS & TAKE_PROFIT
STRATEGY "MomentumBreakout" {
    INDICATOR sma_20 = SMA(CLOSE, 20)
    INDICATOR rsi_14 = RSI(CLOSE, 14)

    ENTRY:       BUY 100 SHARES OF "AAPL"
                 IF CLOSE > sma_20 AND rsi_14 > 50
    EXIT:        SELL ALL
                 IF CLOSE < sma_20
    STOP_LOSS:   SELL ALL
                 IF PRICE < ENTRY_PRICE * 0.96
    TAKE_PROFIT: SELL ALL
                 IF PRICE > ENTRY_PRICE * 1.08
}
```

PHASE 2 REVIEW OBJECTIVES & DELIVERABLES

- Front-to-back compilation: .strat specification executes against real historical data.
- Scoped Symbol Table: Parent-pointer tree with strict cross-strategy scope isolation.
- Semantic Analysis: Two-type system (NUMERIC, BOOLEAN) with 4 compile-time error traps.
- Intermediate Representation: Three-Address Code (TAC) with short-circuit branching.
- Backtest Execution Engine: Quantitative proof of correctness (win rate, return %, drawdown).

Phase 1 (Prototype) vs. Phase 2 (Core Implementation)

Transforming a preliminary syntactic prototype into an end-to-end executable compiler toolchain.

LEXER & PARSER
Full LL(1) Grammar
30 Token Types & AST Hierarchy

SCOPED SYMBOL TABLE
Hierarchical Scopes
Global Builtins + Strategy Scopes

SEMANTIC VALIDATION
Type & Invariant Checks
NUMERIC/BOOLEAN & 4 Error Traps

EXECUTION ENGINE
Backtest Simulation
TAC Quadruples + 63 Passing Tests

01 Lexical Analysis

✕ Phase 1: Basic keyword/literal token scanning without line/col tracking.

✓ **Phase 2: Maximal-munch DFA scanner, 30 token types, exact (line, col) tracking, LexError recovery with source position highlighting.**

02 Syntax Analysis & AST

✕ Phase 1: Single-strategy prototype parser handling basic ENTRY rule.

✓ **Phase 2: Full LL(1) recursive-descent parser, multi-strategy files, indicator decls, ENTRY/EXIT/STOP_LOSS/TAKE_PROFIT rules, precedence climbing.**

03 Symbol Table Management

✕ Phase 1: Not implemented (grammar prototype only).

✓ **Phase 2: Hierarchical parent-pointer symbol table (Global + Strategy child scopes), scope isolation guarantees, forward reference support.**

04 Semantic Analysis & Typing

✕ Phase 1: Not implemented.

✓ **Phase 2: Two-type system (NUMERIC, BOOLEAN), indicator signature & period validation, compile-time detection of undeclared vars, /0, invalid periods.**

05 Intermediate Code (IR)

✕ Phase 1: Not implemented.

✓ **Phase 2: Three-Address Code (TAC) Quadruples with temporaries (t0..tn), jump labels (L0..Ln), and boolean short-circuit branching.**

06 Execution Engine & Testing

✕ Phase 1: Manual testing, 27 prototype unit tests.

✓ **Phase 2: Bar-by-bar interpreter, vector math (SMA/EMA/RSI), portfolio state machine, backtest report, 63 automated tests (100% pass rate).**

Compiler System Architecture: Staged Pipeline

Strict module boundaries and unidirectional dataflow from source text to backtest analytics.

COMPLETED

01

Lexer

lexer.py

.strat → Tokens

COMPLETED

02

Parser

parser.py

Tokens → AST

PHASE 2

03

Symbol Table

symbol_table.py

AST → Scoped Table

PHASE 2

04

Semantic Analyzer

semantic.py

Type & Domain Checks

PHASE 2

05

IR Generator

ir.py

AST → TAC Quadruples

PHASE 2

06

Interpreter

interpreter.py

TAC + Data → Trades

PHASE 2

07

Reporter

interpreter.py

Trades → Analytics

GUARANTEE 1: STRICT UNIDIRECTIONAL FLOW

Each module consumes solely the output of the prior stage with zero cross-module state leakage or hidden dependencies.

GUARANTEE 2: HEADLESS TESTABILITY

The parser operates on in-memory tokens; semantic analysis requires no market data; every phase is tested in absolute isolation.

GUARANTEE 3: COMPILE-TIME VS. RUNTIME DECOUPLING

The symbol table records compile-time formulas and types; the interpreter binds dynamic prices and live rolling indicators.

Lexical Analysis & Recursive-Descent AST

Deterministic maximal-munch scanner and LL(1) recursive-descent parser producing frozen AST nodes.

FRONT-END ARCHITECTURE & DESIGN HIGHLIGHTS

- 30 Distinct Token Types: Keywords, price fields (CLOSE, OPEN, PRICE), operators, and delimiters.
- Exact Source Position: Every token retains (line, column) origin for precise diagnostics.
- Precedence Climbing: Resolves arithmetic (*, /, +, -) and boolean (AND, OR) without left-recursion.
- Immutable Typed AST: Built with Python @dataclass(frozen=True) rooted at ASTNode base.

swayam@vit: ~/Compiler_Design_Proj — Step 1: Lexical Tokens & Position Tracking

```
$ python3 demonstrate_phase2.py --step 1

--------------------------------------------------------------------------------
 DEMO STEP 1: LEXICAL ANALYSIS (Token Stream & Classification)
--------------------------------------------------------------------------------
Total tokens generated: 66
Sample tokens (first 10):
  Token(STRATEGY, 'STRATEGY', line=2, col=1)
  Token(STRING_LIT, 'MomentumBreakout', line=2, col=10)
  Token(LBRACE, '{', line=2, col=29)
  Token(INDICATOR, 'INDICATOR', line=3, col=5)
  Token(IDENT, 'sma_20', line=3, col=15)
  Token(EQ, '=', line=3, col=22)
  Token(SMA, 'SMA', line=3, col=24)
  Token(LPAREN, '(', line=3, col=27)
  Token(CLOSE, 'CLOSE', line=3, col=28)
  Token(COMMA, ',', line=3, col=33)
```

swayam@vit: ~/Compiler_Design_Proj — Step 2: Recursive-Descent AST Generation

```
$ python3 demonstrate_phase2.py --step 2

--------------------------------------------------------------------------------
 DEMO STEP 2: SYNTAX ANALYSIS & AST GENERATION
--------------------------------------------------------------------------------
Pretty-printed Abstract Syntax Tree (AST):
Program (strategies=1)
  Strategy(name='MomentumBreakout')
    indicators:
      IndicatorDecl(name='sma_20')
        FuncCall(name='SMA')
          Ident(CLOSE)
          NumberLit(20)
      IndicatorDecl(name='rsi_14')
        FuncCall(name='RSI')
          Ident(CLOSE)
          NumberLit(14)
    rules:
      RuleBlock(kind='ENTRY')
        action:
          Action(action='BUY', qty=100.0, ticker='AAPL')
        condition:
          BoolOp(op='AND')
            Comparison(op='>')
              Ident(CLOSE)
              Ident(sma_20)
            Comparison(op='>')
              Ident(rsi_14)
              NumberLit(50)
      RuleBlock(kind='EXIT')
        action:
          Action(action='SELL', qty=ALL)
        condition:
          Comparison(op='<')
            Ident(CLOSE)
            Ident(sma_20)
      RuleBlock(kind='STOP_LOSS')
        action:
          Action(action='SELL', qty=ALL)
        condition:
          Comparison(op='<')
            Ident(PRICE)
            BinOpExpr(op='*')
              Ident(ENTRY_PRICE)
              NumberLit(0.96)
      RuleBlock(kind='TAKE_PROFIT')
        action:
          Action(action='SELL', qty=ALL)
        condition:
          Comparison(op='>')
            Ident(PRICE)
            BinOpExpr(op='*')
              Ident(ENTRY_PRICE)
              NumberLit(1.08)
```

Hierarchical Scopes & Lexical Scoping Guarantees

Parent-pointer tree enforcing cross-strategy scope isolation and built-in symbol resolution.

LEXICAL SCOPING MODEL & HIERARCHY

Root Global Scope

Pre-populated with built-in indicator signatures: SMA(series, period), EMA(series, period), RSI(series, period), and 7 price series identifiers (CLOSE, OPEN, HIGH, LOW, VOLUME, PRICE, ENTRY_PRICE).

Strategy Child Scopes

Each STRATEGY block instantiates its own dedicated Scope object pointing to Global as its parent. Stores declared indicators (e.g. sma_20, rsi_14).

Scope Isolation Guarantee

Scope lookup walks strictly outward to the parent Global scope, never sideways to siblings. Identifiers in Strategy A cannot leak into Strategy B.

Forward References Supported

A pre-pass over indicator declarations populates strategy symbols before rule analysis, allowing rules to reference indicators declared anywhere in the block.

```
swayam@vit: ~/Compiler_Design_Proj — Step 3: Scoped Symbol Table & Semantic Invariants

$ python3 demonstrate_phase2.py --step 3

--------------------------------------------------------------------------------
 DEMO STEP 3: SCOPED SYMBOL TABLE & SEMANTIC ANALYSIS
--------------------------------------------------------------------------------
Scoped Symbol Table (Hierarchical Scopes):
+------------------+-------------+-------------+-----------+-------------------------------------+
| Scope            | Symbol Name | Kind        | Data Type | Details / Signature                 |
+------------------+-------------+-------------+-----------+-------------------------------------+
| global           | CLOSE       | PRICE_FIELD | SERIES    | Closing price of current bar        |
| global           | OPEN        | PRICE_FIELD | SERIES    | Opening price of current bar        |
| global           | HIGH        | PRICE_FIELD | SERIES    | High price of current bar           |
| global           | LOW         | PRICE_FIELD | SERIES    | Low price of current bar            |
| global           | VOLUME      | PRICE_FIELD | SERIES    | Trading volume of current bar       |
| global           | PRICE       | PRICE_FIELD | SERIES    | Current price (alias to CLOSE)      |
| global           | ENTRY_PRICE | PRICE_FIELD | SERIES    | Executed entry price of active trade |
| global           | SMA         | BUILTIN_FUNC | SERIES   | args=(SERIES, INT) -> SERIES        |
| global           | EMA         | BUILTIN_FUNC | SERIES   | args=(SERIES, INT) -> SERIES        |
| global           | RSI         | BUILTIN_FUNC | SERIES   | args=(SERIES, INT) -> SERIES        |
+------------------+-------------+-------------+-----------+-------------------------------------+
| MomentumBreakout | sma_20      | INDICATOR   | SERIES    | SMA(CLOSE, 20)                      |
| MomentumBreakout | rsi_14      | INDICATOR   | SERIES    | RSI(CLOSE, 14)                      |
+------------------+-------------+-------------+-----------+-------------------------------------+

Semantic Analysis Status:
  Errors:   0
  Warnings: 0
  Validation: ALL SEMANTIC & TYPE INVARIANTS PASSED
```

Static Type Inference & Indicator Validation

Statically verifying domain constraints and expression type consistency before code generation.

01 Two-Type System

NUMERIC vs. BOOLEAN

- NUMERIC: Price series, scalar numbers, and arithmetic (+, -, *, /).
- BOOLEAN: Relational comparisons (>, <, >=, <=, ==) and logical operators (AND, OR).
- Strict Type Safety: Prevents illegal expressions such as (CLOSE > 100) + 5 at compile time.

02 Indicator Signature Checks

Arity & Argument Type Enforcement

- Function Existence: Must resolve to BUILTIN_FUNC in Global Scope (SMA, EMA, RSI).
- Arity Verification: Requires exactly 2 arguments (series, period).
- Type Integrity: Arg 1 must be a PRICE_SERIES; Arg 2 must be an integer literal > 0.

03 Identifier Resolution

Hierarchical Scope Walking

- Every variable reference in condition trees is resolved against strategy Scope.
- Unresolved identifiers raise immediate SemanticError with exact source coordinates.
- Shadowing Prevention: Reserved series cannot be overridden as variable names.

04 Static Dead-Code Warnings

Unused Indicator Tracking

- After AST traversal, any declared indicator with zero symbol lookups is flagged.
- Emits non-fatal compile-time warnings: [WARNING] Unused indicator 'x'.
- Preserves compilation pipeline while notifying the trader of redundant computations.

Compile-Time Semantic Error Detection

Trapping domain-specific illegal constructs at compile-time with source-anchored diagnostics.

Undeclared Identifier

Typo / Missing Binding

Catches missing indicator declarations or misspelled variables before execution.

Division by Zero

Literal Zero Divisor

Statically evaluates literal divisors (expr / 0) preventing runtime crash.

Invalid Period

Window Period <= 0

Enforces mathematical indicator constraints (e.g. SMA(CLOSE, 0) rejected).

ENTRY_PRICE in ENTRY

Premature Trade State

Disallows referencing trade entry price before a position is actually opened.

swayam@vit: ~/Compiler_Design_Proj — Step 4: Semantic Error Detection (Negative Testing)

```
$ python3 demonstrate_phase2.py --step 4

--------------------------------------------------------------------------------
 DEMO STEP 4: SEMANTIC ERROR DETECTION (Negative Testing)
--------------------------------------------------------------------------------

Testing: Undeclared Identifier
  [SUCCESS] Caught expected SemanticError in strategy 'Err1': Undeclared identifier 'unknown_ind'

Testing: Division by Zero
  [SUCCESS] Caught expected SemanticError in strategy 'Err2': Division by zero in arithmetic expression

Testing: Invalid Indicator Period
  [SUCCESS] Caught expected SemanticError in strategy 'Err3': Period argument to 'SMA' must be a positive integer > 0, got 0

Testing: ENTRY_PRICE used in ENTRY rule
  [SUCCESS] Caught expected SemanticError in strategy 'Err4': ENTRY_PRICE cannot be referenced in an 'ENTRY' rule (no trade position exists before entry
```

Three-Address Code (TAC) Quadruples & Control Flow

Lowering hierarchical AST logic into a linear, machine-independent execution stream.

TAC QUADRUPLE SPECIFICATION & SHORT-CIRCUIT FLOW

Quadruple Structure

Instructions emitted as (op, arg1, arg2, result). Linearizes nested arithmetic and boolean expressions bottom-up.

Register & Label Pools

Monotonically increasing TempPool emits virtual registers t0, t1, t2...; LabelPool emits branch targets L0, L1, L2...

Short-Circuit Boolean Lowering

Boolean AND and OR expressions are lowered using conditional branch quadruples (if not t GOTO Lx), skipping evaluation of secondary operands when false.

First-Class Trading Actions

Dispatches high-level order directives directly in the instruction stream: ACTION [ENTRY] BUY 100.0 SHARES OF 'AAPL'.



Tree-Walking Interpreter & Mathematical Engine

Dynamic environment binding and vector technical indicator calculation algorithms.

01 Dynamic Environment Binding

Bar-by-Bar Runtime State

- At every historical bar, a fresh Environment mapping is initialized with current bar prices (OPEN, HIGH, LOW, CLOSE, VOLUME).
- Dynamic Variable Binding: ENTRY_PRICE is dynamically bound only when an open position exists.
- Temporaries and indicator calculations are cached per bar to avoid redundant re-evaluations.

02 Vector Technical Indicators

High-Performance Incremental Math

- SMA (Simple Moving Average): Sliding-window arithmetic mean with O(1)-amortized deque running sum.
- EMA (Exponential Moving Average): Multiplier k = 2/(N+1), seeded with N-bar SMA window.
- RSI (Relative Strength Index): Wilder's exponential smoothing tracking average gains and losses.

03 Portfolio State Machine

Realistic Order Simulation

- Tracks cash balance, active Position (NONE, LONG, SHORT), and realized profit & loss.
- Order Dispatched on Signals: BUY, SELL, STOP_LOSS, TAKE_PROFIT.
- Sizing Rules: Fixed share quantity or full liquidation (SELL ALL).

04 Quantitative Correctness Oracle

End-to-End Compiler Validation

- The interpreter serves as an executable specification test for every earlier compiler stage.
- A semantic or IR lowering defect manifests immediately as an anomalous trade or metric.
- Ensures formal mathematical rigor across the entire language toolchain.

Backtest Execution Engine & Visual Performance

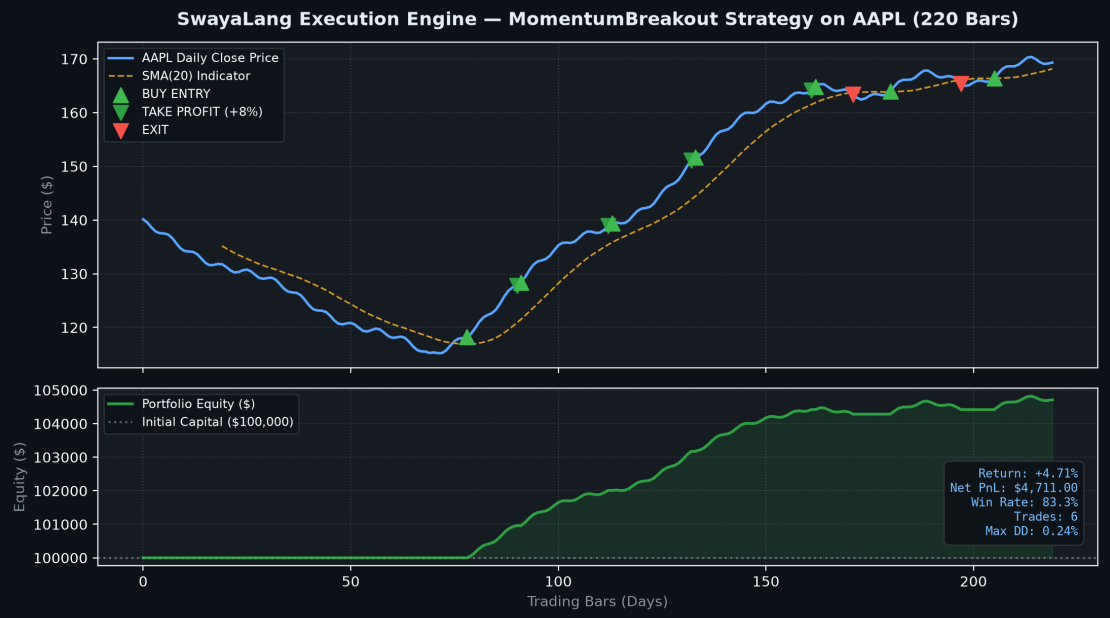
Executing compiled strategy against real historical AAPL data (220 trading bars).

swayam@vit: ~/Compiler_Design_Proj — Step 6: Interpreter Backtest Execution & Trade Log

```
$ python3 demonstrate_phase2.py --step 6

--------------------------------------------------------------------------------
 DEMO STEP 6: EXECUTION ENGINE / INTERPRETER (Market Simulation)
--------------------------------------------------------------------------------
Loaded historical dataset: examples/data/AAPL_daily.csv (220 trading days)
+-------------------------------------------------------------+
|      BACKTEST EXECUTION REPORT: MomentumBreakout (AAPL)      |
+-------------------------------------------------------------+
| Simulation Bars:                220 | Trades Executed:             6 |
| Initial Capital:        $100,000.00 | Winning Trades:              5 |
| Final Equity:           $104,711.00 | Losing Trades:               1 |
| Net Profit / Loss:        $4,711.00 | Win Rate:                83.3% |
| Total Return:                +4.71% | Max Drawdown:            0.24% |
+-------------------------------------------------------------+

Trade Execution Log:
  ID | Date       | Action      | Shares | Price    | Realized PnL | Return %
  ---+------------+-------------+--------+----------+--------------+---------
  01 | 2025-04-13 | BUY_ENTRY   |    100 | $ 118.22 |            - |       -
  02 | 2025-05-03 | SELL_TAKE_PROFIT |    100 | $ 127.82 |     $+960.00 |   +8.12%
  03 | 2025-05-04 | BUY_ENTRY   |    100 | $ 128.37 |            - |       -
  04 | 2025-06-03 | SELL_TAKE_PROFIT |    100 | $ 138.88 |    $+1051.00 |   +8.19%
  05 | 2025-06-04 | BUY_ENTRY   |    100 | $ 139.43 |            - |       -
  06 | 2025-07-01 | SELL_TAKE_PROFIT |    100 | $ 151.07 |    $+1164.00 |   +8.35%
  07 | 2025-07-02 | BUY_ENTRY   |    100 | $ 151.66 |            - |       -
  08 | 2025-08-08 | SELL_TAKE_PROFIT |    100 | $ 164.12 |    $+1246.00 |   +8.22%
  09 | 2025-08-09 | BUY_ENTRY   |    100 | $ 164.81 |            - |       -
  10 | 2025-08-18 | SELL_EXIT   |    100 | $ 163.43 |     $-138.00 |   -0.84%
  11 | 2025-09-05 | BUY_ENTRY   |    100 | $ 163.98 |            - |       -
  12 | 2025-09-22 | SELL_EXIT   |    100 | $ 165.33 |     $+135.00 |   +0.82%
  13 | 2025-10-08 | BUY_ENTRY   |    100 | $ 166.38 |            - |       -
```



Multi-Strategy Compilation & Lexical Scope Isolation

Compiling multi-strategy files with identical variable names in separate scopes.

MULTI-STRATEGY COMPILATION ARCHITECTURE

Multi-Strategy Source Support

A single SwayaLang (.strat) file can define multiple distinct STRATEGY declarations (e.g. FastSMA and SlowSMA in multi_strategy.strat).

Independent Child Scopes

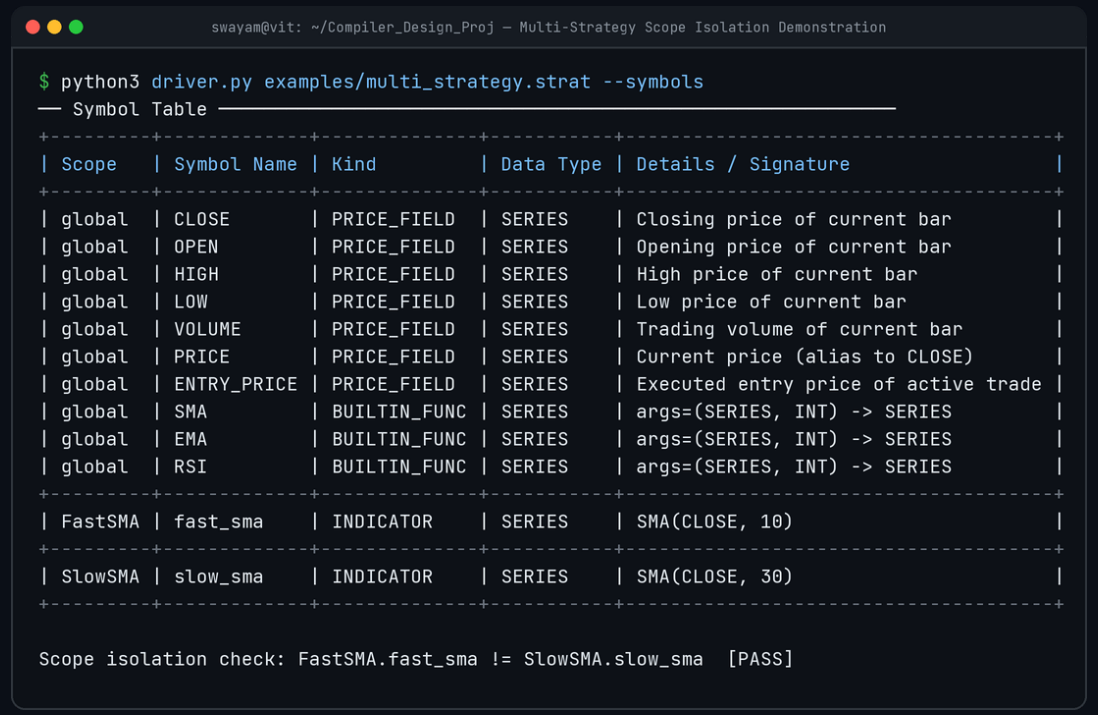
Each strategy definition receives an isolated child scope rooted in the Global Scope. Local symbols are scoped strictly to their respective strategies.

Lexical Collision Prevention

Both FastSMA and SlowSMA can declare independent indicator variables without namespace pollution or cross-strategy state bleeding.

End-to-End Independent Execution

The interpreter walks each StrategyNode independently, producing separate trade logs, portfolio metrics, and equity curves.



Demonstration & Verification — 63 Tests Passing

Exhaustive automated testing across all 8 compiler and runtime modules (100% pass rate).

test_lexer.py (11 Cases)
Token classification, line/col accuracy, LexErrors

test_parser.py (6 Cases)
LL(1) grammar, operator precedence, syntax errors

test_ast.py (4 Cases)
AST node creation, frozen immutability, pretty-print

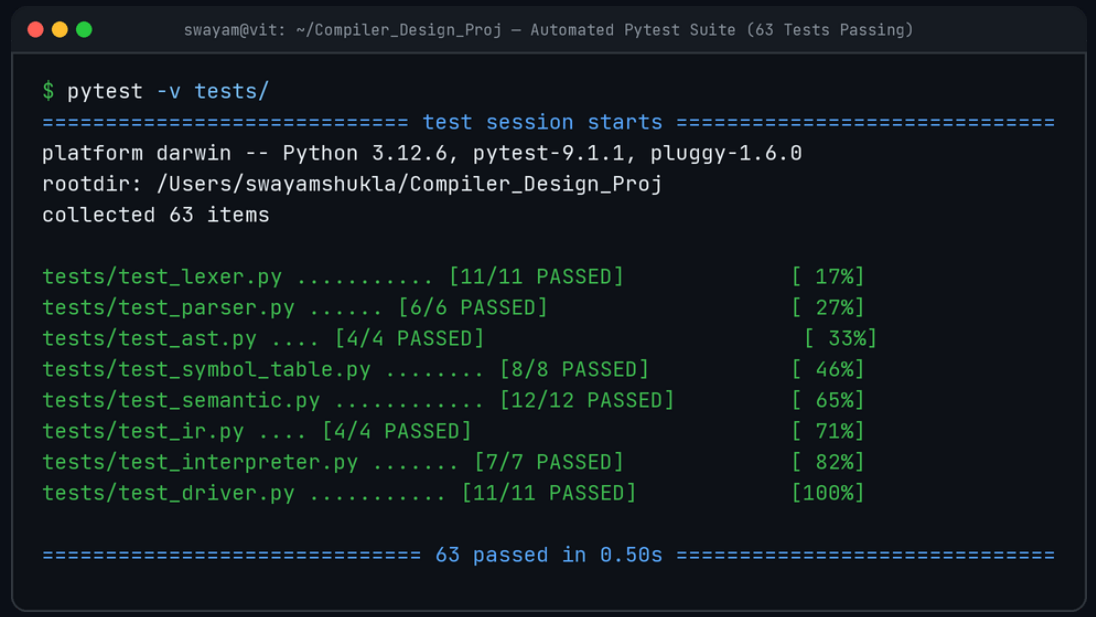
test_symbol_table.py (8 Cases)
Parent-pointer scopes, define/resolve, isolation

test_semantic.py (12 Cases)
Indicator signatures, 2-type rules, 4 error traps

test_ir.py (4 Cases)
TAC quadruples, pools, short-circuit branch flow

test_interpreter.py (7 Cases)
Vector math (SMA/EMA/RSI), portfolio transitions

test_driver.py (11 Cases)
End-to-end CLI flags, exit codes, output dumps



Principal Engineering Challenges & Innovations

Key architectural hurdles resolved during Phase 2 compiler implementation.

Challenge 1: Compile-Time Declaration vs. Runtime State

Indicators are declared once in source code (e.g. SMA(CLOSE, 20)) but must be evaluated incrementally across hundreds of bars.

Resolution: Kept the symbol table strictly compile-time (storing function signature and formula), while the interpreter instantiates stateful running-sum deques and smoothing accumulators at runtime.

Challenge 2: Contextual Keywords vs. Identifier Scoping

Price fields like CLOSE and PRICE act as reserved keywords in indicators but values in arithmetic expressions, creating shadowing risks.

Resolution: Classified them as dedicated PRICE_FIELD tokens in the lexer (never IDENTIFIER), preventing accidental user shadowing (e.g. INDICATOR CLOSE = ...) at the grammar level.

Challenge 3: Linearizing Booleans with Short-Circuiting

Evaluating compound expressions (AND, OR) naively would trigger costly indicator evaluations even when the left operand is false.

Resolution: Lowered boolean combinators into conditional branch quadruples (if not t GOTO Lx), providing genuine short-circuit evaluation directly at the TAC intermediate representation.

Innovation: End-to-End Backtest as Verification Oracle

Compiler unit tests often stop at AST or TAC shape matching, missing subtle runtime semantic discrepancies.

Resolution: Closed the loop with a simulated trading engine over real market data. Any semantic or IR code generation error immediately manifests as an invalid trade log or math anomaly.

Project Roadmap & Phase 3 Execution Timeline

Structured milestones from core implementation to code optimizations and final deliverables.

Phase 1

COMPLETED

Problem Definition & Front-End Prototype

- ✓ Formal EBNF grammar design & project scaffold
- ✓ Hand-written maximal-munch scanner & LL(1) parser
- ✓ Immutable AST node hierarchy & pretty-printer
- ✓ Prototype driver script & 27 unit tests passing

Phase 2

REVIEW 2

Core Compiler & Execution Engine

- ✓ Scoped Symbol Table across multi-strategy files
- ✓ Semantic Analyzer with 2-type system & 4 error traps
- ✓ Three-Address Code (TAC) with short-circuit branching
- ✓ Bar-by-bar Interpreter & vector indicator algorithms
- ✓ Portfolio state machine & historical backtesting
- ✓ 63 automated pytest cases (100% pass rate)

Phase 3

FINAL REVIEW

Code Optimization & Visualization

- ▸ Constant Folding: Evaluate literal sub-expressions at compile time (CLOSE / (10 - 8) → CLOSE / 2)
- ▸ Common Subexpression Elimination (CSE): Deduplicate identical indicator calculations across rules
- ▸ Automated Equity Curve Plotting: Exporting matplotlib charts alongside JSON trade logs
- ▸ Interactive Strategy Web Dashboard & Benchmarks
- ▸ Final Comprehensive Project Report